

SUSE Observability

Procedimento de implantação

Helm e Fleet • versão 6 • setembro de 2026

Cinco dashboards, 33 monitores e três charts com configuração compartilhada. Material em português; dashboards e README em inglês.

Use os arquivos Markdown do pacote para copiar comandos sem quebras visuais. Preencha caminhos, contextos, endpoints, nomes de integração e credenciais próprias antes da implantação.

O relatório tests/RESULTADOS.md e a matriz tests/MATRIZ-SOLICITACOES.md distinguem evidências do laboratório, limitações e aceite necessário em outro cliente. Este pacote customizado não substitui o suporte e o dimensionamento oficial do produto.

SUSE Observability — implantação com Helm e Fleet

1. Escopo e ponto de partida

Revisão v6 · 11 de setembro de 2026. Esta versão reúne os cinco dashboards e os 33 monitores em um único chart de conteúdo. Mantém dois charts de coleta: plataforma e KubeVirt. O ambiente já precisa ter SUSE Observability e o Agent oficial. Instalar estes charts não instala o servidor nem instrumenta automaticamente aplicações.

O laboratório foi validado em 11/09/2026: servidor 2.10.2, CLI 3.3.6, Fleet 0.15.4 e os três clusters abaixo. Os resultados detalhados e as limitações ficam em `tests/RESULTADOS.md`. Uma implantação em outro cliente ainda requer aceite no destino.

Use Helm manual ou Fleet para cada release. Os mesmos arquivos `charts/*/config/*.yaml` servem aos dois. No Fleet, `valuesFiles` carrega explicitamente `values.yaml` primeiro e o perfil do ambiente depois; mantenha essa ordem. Não deixe os dois gerenciadores concorrendo. Os Secrets existentes do laboratório são reutilizados; em um cliente, crie identidades próprias seguindo `REFERENCIA-OPERACIONAL.md`.

Ordem de implantação

1. Conferir servidor, Agents, nomes de integração e acessos.
2. No central, disponibilizar OTLP; no Harvester, preparar CA e registry.
3. Nos clusters RKE2 com Traefik, habilitar métricas e liberar a coleta nas políticas existentes. No etcd, confirmar o endpoint local de métricas.
4. Editar os quatro values de coleta e o único values de conteúdo.
5. Instalar os coletores e confirmar amostras de todos os alvos.
6. Instalar o chart central de conteúdo, conferir Jobs, filtros e monitores.
7. Configurar um canal de notificações e testar abertura e recuperação antes da produção.

Ambientes confirmados

Integração/cluster	Contexto	Namespace do Agent
harvester	harvester	suse-observability
observability-cluster	observability	suse-observability-agent
rancher-kaio	rancher-kaio	suse-observability

No cluster `observability`, servidor SUSE e Jobs de publicação ficam em `suse-observability`. O Agent e a telemetria desse mesmo cluster ficam em `suse-observability-agent`. Isso é intencional.

Quem coleta e quem publica

O Agent oficial fornece topologia Kubernetes e consumo de CPU/memória dos containers. O inventário de pods deste pacote lê requests, limits e estado dos containers diretamente da API Kubernetes, inclusive pods hotplug que não tinham essas séries na coleta instalada. `sre-platform-telemetry` acrescenta `host/etcd/Traefik` e envia Prometheus remote write. `sre-kubevirt-telemetry` executa um Collector OpenTelemetry que lê métricas KubeVirt por HTTPS e envia OTLP gRPC. O único chart `suse-observability-content` executa dois Jobs separados que publicam dashboards e monitores pela API SUSE. Instalar um dashboard não instala nem instrumenta suas fontes de dados. [R1-R4]

2. Pré-requisitos gerais

Na estação: Helm 3, kubectl, acesso aos kubeconfigs e jq para inspeções/copiar a credencial de registry. Bash é usado nos pequenos blocos com variáveis; não é necessário Python local, pip, venv ou CLI sts para a implantação diária. A CLI sts só é necessária para preparar credenciais administrativas ou exportar objetos; os Jobs possuem seu próprio executável.

Passo de preparação: abra a pasta extraída e preencha os caminhos/contextos do cliente. Os nomes de integração usados pelos perfis devem ser substituídos pelos nomes reais, conforme a seção 6. Os namespaces padrão estão indicados em cada comando; ajuste-os se o cliente adotar outra organização:

```
cd /CAMINHO/suse-observability-helm-fleet-v6
export HRV_KCFG=/CAMINHO/harvester.yaml
export OBS_KCFG=/CAMINHO/observability.yaml
export RAN_KCFG=/CAMINHO/rancher.yaml
export HRV_CTX=harvester
export OBS_CTX=observability
export RAN_CTX=rancher-kaio
helm version --short
kubectl get --kubeconfig "$HRV_KCFG" --context "$HRV_CTX" nodes
kubectl get --kubeconfig "$OBS_KCFG" --context "$OBS_CTX" nodes
kubectl get --kubeconfig "$RAN_KCFG" --context "$RAN_CTX" nodes
```

Antes de qualquer alteração, verifique as releases em cada contexto com `helm list -A --kubeconfig ... --kube-context ...`. Não remova servidor, Agent, add-ons, storage, CNI ou ingress controller. Caso um Collector KubeVirt antigo ainda exista, não instale outro em paralelo: mantenha a configuração atual ou faça migração planejada com backup e uma breve lacuna de coleta.

Secrets necessários

Destino	Secret / chave	Ação
Namespace do Agent, em cada cluster	suse-observability-agent-secrets / STS_API_KEY	Reutilizar; não copiar nem alterar pelo projeto.
Harvester, namespace do Agent	application-collection / .dockerconfigjson	Reutilizar se válido ali; é credencial de registry, não de ingestão.
Central, suse-observability	suse-observability-dashboard-token / serviceToken	Reutilizar a credencial proprietária dos dashboards.
Central, suse-observability	suse-observability-monitor-token / serviceToken	Reutilizar a credencial aprovada para monitores.

Um Secret é referenciado pelo Pod no mesmo namespace. Os charts não possuem `secretNamespace` para buscar credenciais de outro namespace. Um token de ingestão compatível com `update-metrics` pode autenticar Agent e OTel; isso não concede permissões administrativas de dashboards/monitores. [R4]

Confirme o nome real da integração, sem imprimir tokens. No Harvester:

```
kubectl get --kubeconfig "$HRV_KCFG" --context "$HRV_CTX" -n suse-observability cm
suse-observability-agent-cluster-name -o jsonpath='{.data.STS_CLUSTER_NAME}{"\n"}'
```

Repita no contexto/namespace correto dos demais. Os valores esperados estão na tabela da seção 1. O `clusterName` dos values deve ser igual a esse resultado. O nome do contexto pode ser diferente.

Rede: os coletores precisam alcançar suas fontes e o receptor SUSE. Os Jobs precisam alcançar a API SUSE. O modo padrão dos publicadores também baixa a CLI por HTTPS; para produção restrita/air gap, prepare a imagem descrita na seção 10. Certificados públicos válidos usam a confiança da imagem. CA corporativa confiada no navegador não é automaticamente confiada nos containers.

Preparar os dois publicadores em uma instância nova

Use a CLI sts autenticada por um administrador da instância SUSE correta. O contexto de kubectl não muda o contexto de sts. Confira a URL na configuração da CLI e use `sts --context NOME ...` explicitamente para o preparo administrativo. As operações abaixo só se aplicam se o cliente ainda não possui papéis/tokens aprovados; no laboratório informado, preserve os existentes.

Permissões do papel de dashboards: `get-dashboards`, `create-dashboards`, `update-dashboards`, `delete-dashboards`. O papel do laboratório também possui `get-metrics`, `get-topology` e `get-views`. Permissões do papel de monitores: `get-monitors`, `create-monitors`, `update-monitors`, `delete-monitors`. O comportamento de importação da base 2.10.2 exige as três permissões de escrita para os monitores, conforme projeto anterior. Elas são permissões de instância, não limitadas aos 33 nomes.

Exemplo para criar os papéis em uma instância nova, usando Bash e um contexto sts já administrativo:

```
export SUSE_ADMIN_CONTEXT=admin-cliente
(
set -euo pipefail
sts --context "$SUSE_ADMIN_CONTEXT" rbac create-subject --subject sre-dashboard-publisher
for PERMISSION in get-dashboards create-dashboards update-dashboards delete-dashboards get-metrics
  get-topology get-views; do
  sts --context "$SUSE_ADMIN_CONTEXT" rbac grant \
    --subject sre-dashboard-publisher --permission "$PERMISSION"
done
sts --context "$SUSE_ADMIN_CONTEXT" rbac create-subject --subject sre-monitor-publisher
for PERMISSION in get-monitors create-monitors update-monitors delete-monitors; do
  sts --context "$SUSE_ADMIN_CONTEXT" rbac grant \
    --subject sre-monitor-publisher --permission "$PERMISSION"
done
)
```

Se o subject já existir, confira as permissões em vez de recriar. Se ocorrer 403, interrompa: o chart não resolve RBAC nem propriedade elevando privilégios. Não crie um bootstrap admin token no servidor como atalho.

Crie um Service Token por publicador, com data de expiração aprovada, e guarde no cofre. Exemplo CLI 3.3.6 para dashboards; execute em subshell Bash, sem `set -x`:

```
(
set -euo pipefail
set +x
umask 077
TMP_DIR=$(mktemp -d)
trap 'rm -rf "$TMP_DIR"' EXIT
# Substitua por uma data aprovada antes de executar.
EXPIRATION=2027-09-11
sts --context "$SUSE_ADMIN_CONTEXT" service-token create \
  --name sre-dashboard-helm --roles sre-dashboard-publisher \
  --expiration "$EXPIRATION" --output json > "$TMP_DIR/result.json"
jq -erj '."service-token".token | select(type=="string" and length>0)' \
  "$TMP_DIR/result.json" > "$TMP_DIR/token"
# Guarde a credencial no cofre antes de remover o temporario.
kubectl create secret generic suse-observability-dashboard-token \
  --kubeconfig "$OBS_KCFG" --context "$OBS_CTX" -n suse-observability \
  --from-file=serviceToken="$TMP_DIR/token"
)
```

Para monitores, use role `sre-monitor-publisher`, nome de token `sre-monitor-helm` e Secret `suse-observability-monitor-token`. O campo continua `serviceToken`. Se o formato JSON da CLI for diferente, interrompa; não salve um Secret com valor vazio. `kubectl create` não sobrescreve credenciais existentes.

Os comandos de exemplo removem temporários ao sair; a cópia no cofre deve ser feita durante o preparo, antes de fechar o subshell. Como alternativa, obtenha o token do cofre diretamente para um arquivo temporário protegido e execute apenas o `kubectl create secret`. Não compartilhe a saída com o valor do token e não versione Secrets em base64.

3. Pré-requisito Harvester: Agent oficial

Se o Agent já estiver instalado e saudável, preserve sua instalação e valide-a; não instale outro Agent sobre ela. O link informado instala o SUSE Observability Agent, não o Collector KubeVirt deste pacote. O add-on aparece como Experimental na documentação Harvester v1.8; registre essa condição na avaliação de suporte do cliente. [R1]

Valide o estado atual:

```
kubectl get --kubeconfig "$HRV_KCFG" --context "$HRV_CTX" -n suse-observability pods
kubectl get --kubeconfig "$HRV_KCFG" --context "$HRV_CTX" -n suse-observability secret
suse-observability-agent-secrets -o go-template='{{range $k,$v := .data}}{{printf "%s\n"
$k}}{{end}}'
```

Os componentes do Agent devem estar prontos e a chave STS_API_KEY deve existir. Confira também a integração na interface SUSE; somente haver pods no namespace não comprova coleta saudável.

Somente para um Harvester novo, sem o add-on

Na interface SUSE, crie a integração Kubernetes do Harvester, escolha seu nome estável e obtenha os dados gerados em Generic Kubernetes: Service Token, stackstate.cluster.name e stackstate.url. Instale o manifesto do add-on v1.8 no Harvester de destino:

```
kubectl apply --kubeconfig "$HRV_KCFG" --context "$HRV_CTX" -f
https://raw.githubusercontent.com/harvester/experimental-addons/v1.8/suse-observability-agent/s
use-observability-agent.yaml
```

Na interface Harvester, abra Addons, selecione suse-observability-agent e Edit YAML. Preencha stackstate.apiKey, stackstate.cluster.name e stackstate.url com os dados da sua própria integração. Habilite o add-on pelo menu Enable. Aguarde os Agents ficarem prontos e valide o registro no SUSE. [R1]

Não copie o token/URL ilustrativos da página pública. O exemplo textual do link contém um caminho receiver duplicado; use o endpoint gerado pela instância, com /receiver/stsAgent uma única vez. Não execute também um Helm independente para gerenciar o mesmo Agent: a fonte de gestão aqui é o add-on. Não coloque o YAML com token no Git.

4. Pré-requisitos de KubeVirt e OTLP

4.1 Preparar a entrada OTLP no SUSE central

No SUSE, confirme o StackPack Open Telemetry. Confirme DNS e certificado do hostname otlp-observability.neuhauss.com.br. O servidor precisa expor o receptor gRPC com TLS. Isso é diferente de instalar um Collector no Harvester. [R2]

Se esse endpoint já funciona, preserve-o. Para uma instalação que ainda não o possui, edite prerequisites/otlp-ingress-values.yaml: hostname, ingressClass, Secret TLS e Service real. O exemplo usa Traefik e suse-observability-otel-collector-grpc:4317, conforme documentação SUSE. Mescle esse fragmento nos arquivos mantidos da instalação do servidor. [R2]

```
# Exemplo: use TODOS os values reais que administram sua release.
helm upgrade suse-observability suse-observability/suse-observability \
--version 2.10.2 --kubeconfig "$OBS_KCFG" --kube-context "$OBS_CTX" \
-n suse-observability \
-f /CAMINHO/values-completos-servidor.yaml \
-f prerequisites/otlp-ingress-values.yaml --wait --timeout 15m
```

A versão 2.10.2 acima é a sua base, não uma recomendação de downgrade para clientes. Use a versão efetiva da release e seus arquivos de sizing/affinity/autenticação. Não aplique apenas o fragmento nem atualize o servidor como efeito colateral da coleta. Planeje a janela porque o upgrade pode reconciliar workloads.

Teste a cadeia, hostname e negociação HTTP/2:

```
openssl s_client -connect otlp-observability.neuhauss.com.br:443 \
  -servername otlp-observability.neuhauss.com.br \
  -verify_hostname otlp-observability.neuhauss.com.br \
  -verify_return_error -alpn h2 </dev/null
```

Espera verificação válida e ALPN h2. Isso testa TLS, não a autorização OTLP nem a entrega de métricas. O values do exporter recebe `otlp-observability.neuhauss.com.br:443`, sem `https://` e sem caminho de remote write.

4.2 Preparar a CA interna do KubeVirt

O certificado público do SUSE não substitui a CA interna usada no scrape KubeVirt. Confira Service, EndpointSlices prontos e CA no Harvester:

```
kubectl get --kubeconfig "$HRV_KCFG" --context "$HRV_CTX" -n harvester-system svc
  kubevirt-prometheus-metrics
kubectl get --kubeconfig "$HRV_KCFG" --context "$HRV_CTX" -n harvester-system endpointslices -l
  kubernetes.io/service-name=kubevirt-prometheus-metrics
mkdir -p private
kubectl get --kubeconfig "$HRV_KCFG" --context "$HRV_CTX" -n harvester-system cm kubevirt-ca -o
  jsonpath='{.data.ca-bundle}' > private/kubevirt-ca.pem
openssl x509 -in private/kubevirt-ca.pem -noout -subject -issuer -dates
```

Obtenha a CA deste ambiente, via acesso Kubernetes confiável. Crie a cópia pública no namespace do Agent:

```
kubectl create --kubeconfig "$HRV_KCFG" --context "$HRV_CTX" -n suse-observability configmap
  kubevirt-metrics-ca --from-file=ca-bundle=private/kubevirt-ca.pem --dry-run=client -o yaml |
  kubectl --kubeconfig "$HRV_KCFG" --context "$HRV_CTX" apply -f -
```

Esse ConfigMap não contém chave privada. A cópia não sincroniza sozinha. Depois de rotação, repita a cópia, aumente `kubevirt-ca.revision` no values e reaplique o chart. Os quatro `serverNames` do chart vêm da base anterior do projeto, não são garantia para toda versão. Erro x509 exige verificar SAN e CA; não desabilite TLS. O teste detalhado está na referência operacional.

4.3 Confirmar o imagePullSecret no namespace correto

No seu laboratório o Secret foi copiado para `suse-observability`. Confirme:

```
kubectl get --kubeconfig "$HRV_KCFG" --context "$HRV_CTX" -n suse-observability secret
  application-collection
```

Se ele só existir em `open-telemetry`, copie apenas o objeto necessário, sem annotations/owners antigos, sem mostrar o conteúdo e sem sobrescrever um Secret existente:

```
set -o pipefail
kubectl get --kubeconfig "$HRV_KCFG" --context "$HRV_CTX" -n open-telemetry secret
  application-collection -o json | jq
  '{apiVersion:"v1",kind:"Secret",metadata:{name:"application-collection",namespace:"suse-observability"},type:.type,data:.data}' | kubectl --kubeconfig "$HRV_KCFG" --context "$HRV_CTX"
  create -f -
```

A cópia é somente da credencial do registry; `STS_API_KEY` continua compartilhada diretamente com o Agent. Não delete o namespace antigo: ele pode conter outros recursos. Autenticação para `dp.apps.rancher.io` não comprova autorização para toda imagem; confirme o pull real no rollout do Collector.

5. Pré-requisito RKE2: configurar o Traefik

Faça isto nos clusters `observability-cluster` e `rancher-kaio` se ainda não estiver pronto. No Harvester mantenha `traefik.enabled: false`. O chart complementar não instala nem reconfigura o Traefik. [R5]

No Rancher, para cluster provisionado por ele: Cluster Management → cluster → Edit Config → Edit as YAML. Mescle `prerequisites/rancher-rke2-traefik.yaml` em `spec.rkeConfig.chartValues.rke2-traefik`. Preserve outras portas, logs, TLS e regras existentes.

Para o cluster local/importado, sem esse bloco de provisionamento, use a fonte persistente que administra o HelmChartConfig `rke2-traefik` em `kube-system`. Mescle `prerequisites/rke2-traefik-values.yaml` em

spec.valuesContent. Não instale um segundo Traefik e não crie dois arquivos concorrentes para o mesmo recurso. [R5]

O trecho de values necessário é:

```
metrics:
  prometheus:
    entryPoint: metrics
    addEntryPointsLabels: true
    addServicesLabels: true
ports:
  metrics:
    port: 9100
    expose:
      default: false
```

A alteração pode recriar pods do Traefik; agende uma janela adequada. A porta é interna. Não crie NodePort, LoadBalancer ou Ingress público para 9100. Após a reconciliação, valide um pod no cluster Observability:

```
kubectl get --kubeconfig "$OBS_KCFG" --context "$OBS_CTX" -n kube-system pods -l
  app.kubernetes.io/name=rke2-traefik
POD=$(kubectl --kubeconfig "$OBS_KCFG" --context "$OBS_CTX" \
  -n kube-system get pods -l app.kubernetes.io/name=rke2-traefik \
  -o jsonpath='{.items[0].metadata.name}')
kubectl port-forward --kubeconfig "$OBS_KCFG" --context "$OBS_CTX" -n kube-system "pod/$POD"
19100:9100
```

Em outro terminal: `curl --fail http://127.0.0.1:19100/metrics`. Espere HTTP 200 e amostras `traefik_...`. Encerre com `Ctrl+C`. Repita nas demais réplicas e no Rancher. Esse túnel testa o produtor, não NetworkPolicy entre pods. O `helm test` da seção 7 complementa a verificação de rede após instalar a coleta.

Mantenha `allowMetricsNetworkPolicy: false` por padrão. Se já houver isolamento de ingresso no Traefik e faltar a permissão 9100, revise regras `web/websecure` e habilite também `existingIsolationConfirmed: true`. Criar a primeira política pode bloquear tráfego antes permitido. Regras de egress também precisam ser analisadas. Access logs são opcionais e não são habilitados pelo pacote; o fragmento separado exige decisão de retenção e privacidade.

5.1 Confirmar as métricas locais do etcd

O chart usa um DaemonSet com `hostNetwork` somente nos nós com o label configurado em `etcd.nodeSelector`. O endpoint padrão é `http://127.0.0.1:2381/metrics`, no próprio nó. Não usa a API cliente TLS na porta 2379 nem lê a chave privada do etcd. Nos três clusters testados, esse endpoint já estava disponível; os sete membros foram coletados.

Antes de instalar, confira os nós e os argumentos dos pods estáticos. Exemplo no cluster Observability:

```
kubectl get nodes --show-labels --kubeconfig "$OBS_KCFG" --context "$OBS_CTX"
kubectl get pods -n kube-system -l component=etcd \
  --kubeconfig "$OBS_KCFG" --context "$OBS_CTX" \
  -o jsonpath='{range
  .items[*]}{.metadata.name}{"\n"}{.spec.containers[0].command}{"\n"}{.spec.containers[0].args}{"\n"}{end}'
```

Confirme o endereço em `--listen-metrics-urls` e o label `node-role.kubernetes.io/etcd=true`. No SO de cada nó etcd, `curl --fail http://127.0.0.1:2381/metrics` deve responder com famílias `etcd_...`. Ajuste `etcd.endpoint/seletor` apenas se a configuração real for diferente. Se a distribuição não disponibiliza esse endpoint local, revise a configuração persistente suportada pelo RKE2 e planeje qualquer restart do control plane; não altere manualmente o manifesto estático gerado.

`etcd-expose-metrics` expõe métricas na interface cliente. Não é necessário ativá-lo para esse desenho de coleta local; não abra a porta publicamente nem troque o bind por `0.0.0.0` como atalho. Em um etcd de membro único, como o Harvester deste laboratório, não existe tráfego entre peers e o gráfico de RTT não se aplica. Isso é diferente de falha de scrape. [R11]

6. Preencher os values e validar localmente

Não edite arquivos gerados: eles não existem nesta versão. Edite os values de charts/*/config/ correspondentes ao componente. Os exemplos estão preenchidos para seu laboratório.

Arquivo	Principal ajuste no cliente
charts/sre-platform-telemetry/config/*.yaml	clusterName, remoteWrite.url, Secret do Agent, etcd/Traefik.
charts/sre-kubevirt-telemetry/config/values.yaml	clusterName, endpoint OTLP, Secret do Agent, registry e CA.
charts/suse-observability-content/config/values.yaml	URL SUSE, dois Secrets de publicação e conteúdo selecionado.

O destino Kubernetes não pertence ao values: escolha --kubeconfig, --kube-context e -n no comando, ou defaultNamespace/seletores no Fleet. Um dashboard central não precisa de kubeconfig de todos os clusters para mostrá-los.

Valide os três charts com os cinco perfis sem instalar nada:

```
bash tests/helm-checks.sh
```

Esse teste usa seu Helm local para lint/schema/template, inclusive casos negativos de configuração. Não prova pull de imagens, RBAC no servidor, TLS dos alvos ou ingestão. Para verificar políticas de admissão, renderize o componente e faça `kubectl apply --dry-run=server` no destino antes da instalação. Não aplique o manifesto renderizado de verdade: a gestão será pelo Helm.

7. Instalar os coletores com Helm

7.1 Canário: telemetria no cluster Observability

```
helm upgrade --install sre-platform-telemetry \
  ./charts/sre-platform-telemetry \
  --kubeconfig "$OBS_KCFG" --kube-context "$OBS_CTX" \
  -n suse-observability-agent \
  -f charts/sre-platform-telemetry/config/observability-cluster.yaml \
  --wait --timeout 10m
helm test sre-platform-telemetry \
  --kubeconfig "$OBS_KCFG" --kube-context "$OBS_CTX" \
  -n suse-observability-agent --logs --timeout 3m
```

O teste acessa as réplicas Traefik pelo Service interno, a partir de um Pod no namespace de coleta e com o label da política do coletor. Deve confirmar HTTP 200 e métricas Traefik. Não testa ingestão no SUSE nem o etcd. Confira no Metrics Explorer `up{cluster_name="observability-cluster",job="sre-node"}`, o mesmo para sre-etcd e traefik. Compare a quantidade de séries up com os alvos esperados, não aceite apenas uma amostra saudável.

7.2 Demais clusters, após aceite do canário

```
helm upgrade --install sre-platform-telemetry \
  ./charts/sre-platform-telemetry \
  --kubeconfig "$RAN_KCFG" --kube-context "$RAN_CTX" \
  -n suse-observability \
  -f charts/sre-platform-telemetry/config/rancher-kaio.yaml --wait --timeout 10m
helm test sre-platform-telemetry \
  --kubeconfig "$RAN_KCFG" --kube-context "$RAN_CTX" \
  -n suse-observability --logs --timeout 3m
helm upgrade --install sre-platform-telemetry \
  ./charts/sre-platform-telemetry \
  --kubeconfig "$HRV_KCFG" --kube-context "$HRV_CTX" \
  -n suse-observability \
  -f charts/sre-platform-telemetry/config/harvester.yaml --wait --timeout 10m
```

7.3 Collector KubeVirt no Harvester


```
helm upgrade --install kubevirt-otel-collector \
  ./charts/sre-kubevirt-telemetry \
  --kubeconfig "$HRV_KCFG" --kube-context "$HRV_CTX" \
  -n suse-observability \
  -f charts/sre-kubevirt-telemetry/config/values.yaml --wait --timeout 10m
kubectl logs --kubeconfig "$HRV_KCFG" --context "$HRV_CTX" -n suse-observability
  deploy/kubevirt-otel-collector --since=10m
```

Esse é um chart customizado pequeno e sem dependência do chart upstream. Não usa mais `chartVersion: 0.165.0` dos valores antigos; sua versão própria é 0.2.0 e a imagem-base foi mantida em 0.156.0-k8s-13.1. A imagem foi executada no Harvester durante esta validação. Em outro ambiente, confirme o acesso ao registry e os componentes disponíveis antes do aceite.

Espere um Collector pronto, sem 401/403, x509 ou falhas contínuas de envio. No SUSE:

```
up{service_name="kubevirt-metrics",k8s_cluster_name="harvester"}
kubevirt_vmi_info{k8s_cluster_name="harvester"}
time() - timestamp(kubevirt_vmi_memory_usable_bytes{k8s_cluster_name="harvester"})
```

Confirme todos os alvos esperados e amostras recentes. VM desligada não deve ter consumo atual inventado; algumas métricas guest dependem de balloon/QEMU Guest Agent. As VMs não recebem outro Collector. Um erro de TLS deve ser resolvido validando certificado/CA/SAN, nunca com `insecure_skip_verify: true`.

8. Publicar dashboards e monitores

8.1 Preparar as credenciais uma vez

No seu laboratório, reutilize os dois Secrets existentes. O anexo mostra que a conta humana tinha permissões de dashboards, mas a exclusão só funcionou com o token proprietário. Não substitua esse token apenas por outro com nome ou role semelhante. [E1, R6]

```
kubectl get --kubeconfig "$OBS_KCFG" --context "$OBS_CTX" -n suse-observability secret
  suse-observability-dashboard-token suse-observability-monitor-token
```

Para um cliente sem esses Secrets, um administrador SUSE deve preparar os roles e Service Tokens da referência docs/REFERENCIA-OPERACIONAL.md, guardá-los no cofre e criar os Secrets por arquivo protegido. Não use token de Rancher, token de ServiceAccount Kubernetes ou STS_API_KEY de ingestão como substituto administrativo.

8.2 Editar um arquivo e publicar todo o conteúdo

Edite `charts/suse-observability-content/config/values.yaml`: preencha `observability.url` uma única vez; configure os dois Secrets em `auth.dashboards` e `auth.monitors`. Em `dashboards.include`, escolha entre `applications`, `resources`, `platform`, `traefik` e `harvester`. `monitors.include: []` inclui o catálogo completo; para limitar regras, use os slugs de `charts/suse-observability-content/files/catalog.json`.

```
helm upgrade --install suse-observability-content \
  ./charts/suse-observability-content \
  --kubeconfig "$OBS_KCFG" --kube-context "$OBS_CTX" \
  -n suse-observability -f charts/suse-observability-content/config/values.yaml \
  --wait --timeout 15m
kubectl logs job/suse-observability-content-dashboards \
  --kubeconfig "$OBS_KCFG" --context "$OBS_CTX" -n suse-observability
kubectl logs job/suse-observability-content-monitors \
  --kubeconfig "$OBS_KCFG" --context "$OBS_CTX" -n suse-observability
```

Aceite: ambos os Jobs Complete; cinco dashboards e 33 monitores conferidos na API; uma segunda execução sem mudanças deve informar que os objetos permanecem iguais. `monitors.enabled: false` permite publicar somente dashboards; `dashboards.enabled: false`, somente monitores. Não crie outra release com os mesmos nomes de objetos.

8.3 Migrar da v5

Exporte os dashboards antes da migração, preserve o token proprietário e interrompa a reconciliação dos bundles antigos no Fleet, se existirem. Instale o chart central com os mesmos nomes e Secrets. Somente após os Jobs concluírem, remova as cinco releases antigas `suse-dashboard-*` e a antiga release `suse-observability-monitors`, se instalada. A desinstalação desses charts remove objetos Kubernetes, não as definições já gravadas no SUSE.

O publicador usa JSON original, sem passar o campo `y` pelo parser YAML do Helm. Na v5, esse parser transformou `y` em `true`, eliminando a posição vertical e provocando sobreposição. A comparação também normaliza somente os wrappers equivalentes de variáveis retornados pela CLI. Coordenadas, consultas e filtros continuam sendo verificados integralmente.

Os dashboards usam `saveVariables: false`. Alterar filtros deve mudar a URL sem marcar o dashboard como editado. Edições reais de widgets continuam exigindo salvar. Se uma aba antiga ainda mostrar um asterisco, descarte apenas as alterações de visualização dessa aba e reabra a definição corrigida. Não grave a disposição antiga por cima da versão publicada.

9. Usar os mesmos charts no Fleet

Cada pasta em `charts/` contém seu `fleet.yaml` e seus arquivos `config/`. O Fleet usa `helm.chart: .:` não depende de arquivos fora do bundle. O chart de plataforma possui três `targetCustomizations`, cada um carregando o `values` do seu cluster. São três `GitRepos`: coleta downstream, coleta no Rancher local e conteúdo central. Assim você libera cada fase depois de seus pré-requisitos. [R7]

Primeiro publique o conteúdo desta pasta na raiz de uma branch, por exemplo `helm-fleet-v6`, no repositório desejado. O ZIP não altera seu GitHub. Os `GitRepos` fornecidos apontam para seu repositório e essa branch; ajuste antes de aplicar.

9.1 Selecionar os clusters de destino

No cluster de gerenciamento, descubra os recursos Fleet:

```
kubectl get --kubeconfig "$RAN_KCFG" --context "$RAN_CTX" clusters.fleet.cattle.io -A
--show-labels
```

Adicione nos recursos Fleet correspondentes, pela UI ou por `kubectl label`, as duas labels abaixo. O nome do recurso Fleet pode ser `local` ou `c-...`: não presuma que é igual à integração SUSE. Não aplique as labels nos Nodes.

Cluster de destino	Labels a atribuir ao recurso Fleet
Harvester	<code>observability.example.com/enabled=true</code> e <code>observability.example.com/cluster=harvester</code>
Servidor SUSE	<code>observability.example.com/enabled=true</code> e <code>observability.example.com/cluster=observability-cluster</code>
Rancher	<code>observability.example.com/enabled=true</code> e <code>observability.example.com/cluster=rancher-kaio</code>

Exemplo deliberadamente com placeholders para o recurso real:

```
kubectl label --kubeconfig "$RAN_KCFG" --context "$RAN_CTX" -n <WORKSPACE_FLEET>
clusters.fleet.cattle.io <NOME_REAL> observability.example.com/enabled=true
observability.example.com/cluster=observability-cluster --overwrite
```

O `GitRepo` deve ficar no workspace que contém seus Fleet Clusters. Downstreams e o cluster local podem estar em workspaces diferentes: confira `fleet-default/fleet-local` no seu Rancher e crie a configuração no workspace adequado. Não force todos a um namespace presumido.

9.2 Aplicar por fase

Edite `fleet/gitrepo-collectors.yaml`, confirmando `repo`, `branch`, `paths` e `workspace`. Aplique-o no Kubernetes de gerenciamento e aguarde seus `BundleDeployments` ficarem `Ready`; execute a aceitação das métricas. Use `gitrepo-management.yaml` para a coleta do Rancher local; depois do aceite das fontes, aplique `gitrepo-content.yaml`.

```
kubectl apply --kubeconfig "$RAN_KCFG" --context "$RAN_CTX" -f fleet/gitrepo-collectors.yaml
# Depois do aceite dos coletores:
kubectl apply --kubeconfig "$RAN_KCFG" --context "$RAN_CTX" -f fleet/gitrepo-content.yaml
```

A regra final de cada bundle tem `doNotDeploy: true`: não há implantação nos demais clusters por padrão. Os dashboards/monitores miram apenas `observability-cluster.defaultNamespace` preserva recursos que precisam ir a `kube-system/harvester-system`; não substitua por `namespace`, que importaria um namespace único. `disablePreProcess: true` evita conflitos com expressões `${...}`; `waitForJobs: true` aguarda os publicadores. [R7]

Uma alteração em Git provoca reconciliação. Para repetir uma publicação sem mudar a definição, incremente `lifecycle.revision`. Fleet não verifica continuamente o conteúdo remoto da API SUSE. Alterações manuais somente na UI não são automaticamente corrigidas. Ações externas também não são desfeitas por rollback Helm/Fleet. [R8]

Se você já instalou manualmente e quiser passar para Fleet, não habilite `force/takeOwnership` como atalho. No laboratório, remova apenas as releases customizadas após guardar os values; mantenha `Secrets/CA` e objetos SUSE. Depois deixe Fleet criar as mesmas releases e reconciliar os mesmos objetos, com o mesmo publicador. Essa transição tem pequena lacuna de coleta e deve ser planejada; jamais desinstale o Agent oficial.

Atenção aos workspaces Fleet

No laboratório, `clusters.fleet.cattle.io/observability` e `c-7f8gg` (Harvester) estão em `fleet-default`. O Rancher local está em `fleet-local`, com nome `local`. Um GitRepo em `fleet-default` não seleciona o cluster de `fleet-local`. Por isso há um terceiro manifesto, `fleet/gitrepo-management.yaml`, somente para a coleta do Rancher local.

Aplique as labels nos objetos **Cluster do Fleet no management cluster**, nunca nos Nodes Kubernetes. Exemplos do laboratório:

```
kubectl label clusters.fleet.cattle.io observability -n fleet-default \
  --kubeconfig "$RAN_KCFG" --context "$RAN_CTX" \
  observability.example.com/enabled=true \
  observability.example.com/cluster=observability-cluster --overwrite
kubectl label clusters.fleet.cattle.io c-7f8gg -n fleet-default \
  --kubeconfig "$RAN_KCFG" --context "$RAN_CTX" \
  observability.example.com/enabled=true \
  observability.example.com/cluster=harvester --overwrite
kubectl label clusters.fleet.cattle.io local -n fleet-local \
  --kubeconfig "$RAN_KCFG" --context "$RAN_CTX" \
  observability.example.com/enabled=true \
  observability.example.com/cluster=rancher-kaio --overwrite
```

No cliente, substitua esses nomes pelos retornados por `kubectl get clusters.fleet.cattle.io -A` no management cluster. Revise também `defaultNamespace` em cada `charts/*/fleet.yaml`. Não use namespace para forçar todos os recursos: roles do KubeVirt e Service/NetworkPolicy do Traefik pertencem a outros namespaces.

Depois de publicar esta pasta na branch configurada, aplique também `fleet/gitrepo-management.yaml` no management cluster. O ZIP não cria nem atualiza essa branch automaticamente. Não aplique GitRepos apontando para uma branch que ainda contenha a v5.

10. Segurança, atualização e aceite

O perfil conservador mantém scrape de 30s, timeout de 10s, allowlist de famílias e recursos/filas limitados. observe significa sem os limites opcionais de cardinalidade, **não** ausência de filtros ou de perda possível. Em enforce, sample/label limits podem invalidar o scrape; exceder series_limit do vmagent também pode descartar séries. Meça antes de impor limites e monitore a coleta. As métricas e consultas úteis estão na referência. [R9]

A allowlist reduz o volume enviado, mas não elimina o custo de gerar a resposta no exporter. Ela preserva os dashboards/monitores do pacote; MetricBindings externos podem exigir métricas adicionais. O exemplo examples/kubevirt-extra-metricbindings.values.yaml amplia as famílias do tutorial anterior, sem instalar Views ou MetricBindings automaticamente.

O OTel usa um Deployment, batch máximo de 1.000 pontos, fila de 128 requisições, dois consumidores e retry finito. Requisições não equivalem a bytes: fila limitada não garante um teto exato de memória. O memory_limiter e limits reduzem exposição, mas podem recusar métricas/OOM se subdimensionados. A fila vmagent usa disco efêmero; expira com o Pod. Há consumo real de CPU, RAM, rede e disco, inclusive nos nós de etcd. [R10]

Para produção restrita, construa images/Dockerfile.publisher, fixe imagens de origem e final por digest, publique no registry aprovado e mescle examples/publisher-prebuilt.values.yaml no values central. Ajuste publisher.imagePullSecrets. Isso evita baixar sts a cada Job. O modo padrão é para teste com egress HTTPS e checksum SHA-256 fixado; **tag fixa não é digest imutável**. Espelhe também node-exporter, vmagent, Python do inventário/teste e Collector OTel. O pacote sozinho não é air-gapped.

Antes de upgrades: guarde valores da release, versões/digests, exports dos objetos SUSE e inventário de alvos. Depois: confira novamente métricas Traefik, etcd, KubeVirt, SAN/CA, filas e queries. O publicador faz preflight dos objetos selecionados; guarde exports externos antes de substituições. O chart valida configuração local; o aceite de runtime continua necessário.

Critérios mínimos: alvos esperados com up=1 e dados recentes; nenhum erro contínuo de autenticação/TLS; fila drenando sem descartes sustentados; sem pressão/reinícios causados pelos coletores; cinco telas conferidas nos três filtros de cluster; monitores esperados persistidos e com escopo correto. Configure notificações e faça teste de entrega separado. Pods Ready e Jobs Completed não comprovam, sozinhos, todos esses itens.

11. Remoção controlada

helm uninstall remove os recursos geridos pela release, mas não deve apagar automaticamente dashboards/monitores na API SUSE. Hooks podem permanecer até a política de limpeza/TTL. Para remover apenas uma tela, primeiro execute um plano com a mesma credencial proprietária:

```
helm upgrade --install suse-observability-content ./charts/suse-observability-content \
  --kubeconfig "$OBS_KCFG" --kube-context "$OBS_CTX" -n suse-observability \
  -f charts/suse-observability-content/config/values.yaml \
  --set dashboards.include={traefik} --set monitors.enabled=false \
  --set lifecycle.action=plan --wait --timeout 10m
kubectl logs --kubeconfig "$OBS_KCFG" --context "$OBS_CTX" -n suse-observability
  job/suse-observability-content-dashboards
```

Revise o nome/ID. Para excluir somente essa definição:

```
helm upgrade --install suse-observability-content ./charts/suse-observability-content \
  --kubeconfig "$OBS_KCFG" --kube-context "$OBS_CTX" -n suse-observability \
  -f charts/suse-observability-content/config/values.yaml \
  --set dashboards.include={traefik} --set monitors.enabled=false \
  --set lifecycle.action=delete \
  --set lifecycle.confirmDelete=DELETE_MANAGED_OBJECTS --wait --timeout 10m
# Somente se o Job de delete terminar com sucesso:
helm uninstall suse-observability-content \
  --kubeconfig "$OBS_KCFG" --kube-context "$OBS_CTX" -n suse-observability
```

O exemplo acima seleciona somente Traefik e desabilita a ação em monitores. Para monitores, a ação mira o catálogo selecionado por monitors.include, recusando divergência de identifier/tags. Diminuir a lista ou

desinstalar um chart não apaga por inferência objetos externos. Preserve nativos e tokens. `plan` não prova permissão de escrita; um 403 no delete deve ser resolvido com a identidade autorizada, nunca criando duplicatas ou removendo todos os objetos.

No Fleet, faça a mesma ação pelos values em Git: `action: delete` + confirmação, aguarde sucesso, então remova o `bundle/path`. Não execute um Helm manual concorrente. Para voltar a publicar, retorne explicitamente a `action: apply` e confirmação vazia.

12. Depois do aceite

Para estudar o funcionamento e criar suas próprias métricas, dashboards e monitores, use `GUIA-ESTUDO.md` (`GUIA-ESTUDO.md`). O relatório de recursos e segurança (`IMPACTO-E-SEGURANCA.md`) separa requests, uso medido, crescimento de PVCs, estimativas e limites do laboratório.

Alertas e leitura operacional

Todos os 125 painéis de métricas possuem orientação de investigação no ponto de interrogação. Consulte a matriz completa de cobertura (`ALERT-COVERAGE.md`), que relaciona cada gráfico aos 33 monitores customizados ou aos monitores nativos. Ajuste o limiar ao SLO e ao baseline antes de encaminhar alertas. O limite de 20 ms para disco por dez minutos é uma referência inicial de investigação, com atividade mínima de I/O; não é garantia universal de desempenho.

A fila de telemetria representa bytes aguardando envio ao SUSE Observability: zero é saudável. Uma série ausente não equivale a zero. Os painéis de Pods exibem contagem e identificação; réplicas são apresentadas como cópias faltantes da aplicação. Os discos operacionais excluem partições de boot/runtime e binds repetidos, preservando dados e persistência do Harvester.

A validação de consultas, fontes e publicadores está em `RESULTADOS` (`../tests/RESULTADOS.md`). A inspeção da interface e seus limites estão em `UI-VALIDATION` (`../tests/UI-VALIDATION.md`). O SMTP é configurado globalmente no servidor e as notificações são criadas manualmente. No laboratório, canal, abertura real e recuperação por e-mail foram recebidos e confirmados. Isso não configura nem valida o destino de outro cliente.

Referências e limites da validação

[E1] Anexo do usuário `Pasted text.txt`: roles, uso do Secret proprietário, exclusão dos cinco dashboards e 24 monitores da versão anterior. É evidência do laboratório, não teste dos novos charts.

[R1] Harvester v1.8 — SUSE Observability Agent (Experimental):
<https://docs.harvesterhci.io/v1.8/advanced/addons/suse-observability-agent/>

[R2] SUSE — Exposição de Ingress/OTLP: https://documentation.suse.com/en-us/cloudnative/suse-observability/latest/en/setup/install-stackstate/kubernetes_openshift/ingress.html

[R3] SUSE — Prometheus remote write: <https://documentation.suse.com/cloudnative/suse-observability/latest/en/use/metrics/k8s-prometheus-remote-write.html>

[R4] SUSE — Service tokens: <https://documentation.suse.com/cloudnative/suse-observability/latest/en/use/security/k8s-service-tokens.html>

[R5] RKE2 — HelmChartConfig:
<https://documentation.suse.com/cloudnative/rke2/latest/en/add-ons/helm.html>

[R6] SUSE — Dashboards:
<https://documentation.suse.com/cloudnative/suse-observability/latest/en/use/dashboards/dashboards.html>

[R7] Fleet 0.15 — `fleet.yaml`: <https://fleet.rancher.io/0.15/reference/ref-fleet-yaml>

[R8] Helm — hooks: https://helm.sh/docs/topics/charts_hooks/

[R9] VictoriaMetrics — `vmagent`: <https://docs.victoriametrics.com/vmagent.html>

[R10] OpenTelemetry Collector v0.156.0 — exporterhelper e batch processor:

<https://github.com/open-telemetry/opentelemetry-collector/tree/v0.156.0/exporter/exporterhelper> e

<https://github.com/open-telemetry/opentelemetry-collector/tree/v0.156.0/processor/batchprocessor>

Os testes executados e não executados nesta entrega estão em `tests/RESULTADOS.md`. Houve validação ao vivo dos três clusters, dos Jobs e das APIs SUSE. Consulte as pendências e os limites no relatório; a validação deste laboratório não substitui a homologação do cliente.

[R11] RKE2 — opções do servidor e métricas: https://docs.rke2.io/reference/server_config e

<https://docs.rke2.io/reference/metrics>

[R12] Google SRE — Monitoring Distributed Systems:

<https://sre.google/sre-book/monitoring-distributed-systems/>

[R13] Brendan Gregg — USE Method: <https://www.brendangregg.com/usemethod.html>

[R14] KubeVirt — definições de métricas: <https://kubevirt.io/monitoring/metrics.html>

Comparar um benchmark curto com o dashboard

O navegador calcula a taxa durante a execução do teste. O cálculo de taxa recente (retirado do dashboard para simplificar a leitura) calcula o incremento entre as duas últimas coletas, normalmente 30 segundos. Após o teste, duas coletas sem chamadas novas fazem essa taxa voltar a zero. A média de cinco minutos ainda inclui o teste. Exemplo: 200 chamadas em 1,49 segundo equivalem a cerca de 134 req/s no navegador, 6,67 req/s se concentradas em um intervalo de coleta de 30 segundos, e 0,67 req/s na janela de cinco minutos. Esses números medem janelas diferentes.

Para testar contagens: use uma aplicação de teste, faça uma chamada inicial para cada rota/método/código que será exercitado, aguarde pelo menos duas coletas e a ingestão, registre os counters, execute o teste e aguarde a ingestão antes de comparar os deltas. Uma série nova que aparece inicialmente com valor 10 não prova em qual instante essas dez chamadas aconteceram; `rate/irate` não podem reconstruir um incremento sem amostra anterior. Não faça esse aquecimento com falhas em aplicações de cliente; limite-o à aplicação de teste. Para hora e quantidade exatas de cada request, use `access logs/traces`. O p95 do histograma também é uma estimativa por faixas, não o percentil exato calculado pelo navegador.